

Git Part 1 — Foundations

ICCS121 Syskills | MUIC

Lecture 05 | Version control, repositories, branching, and remote collaboration

Who Am I?

- ผศ.ดร. ทรงพล ตีระกนก (Asst. Prof. Dr. Songpon Teerakanok)
- Assistant Professor, MUIC — Computer Science
- Founder, CYBERSKILLS Professional Training Center
- Research: Cybersecurity, Network Security, AI Security
- Email: songpon.tee@mahidol.ac.th

Disclaimer

- Slides are a **summary** — not a complete reference
- Always verify commands in your own terminal
- Some behavior may differ by OS version or distro
- For authoritative info: `man` pages, official docs

Agenda

- 1.** Why Version Control?
- 2.** Centralized vs. Distributed VCS
- 3.** Introduction to Git
- 4.** Repositories — git init and git clone
- 5.** The Three Areas of Git
- 6.** The Basic Workflow
- 7.** Inspecting Your Repository
- 8.** Branching
- 9.** Remote Repositories + The .gitignore File

1. Why Version Control?

Common horror stories without it

The Horror Stories

- **"My code worked yesterday, but today it is broken"**
Forgot what you changed; cannot undo
- **"I lost all my work"**
Hard drive failed, laptop stolen, folder deleted
- **"My teammate deleted my code"**
Shared folder, someone overwrote your file



src: ai-generated

What Is a Version Control System?

- **Tracks changes** — every modification recorded (who, when, why)
- **Facilitates experimenting** — try ideas on a branch, discard if they fail
- **Enables collaboration** — multiple devs, same project, no overwrites
- **Records contributions** — see who wrote each line ("blame")

2. Centralized vs. Distributed VCS

Two fundamentally different architectures

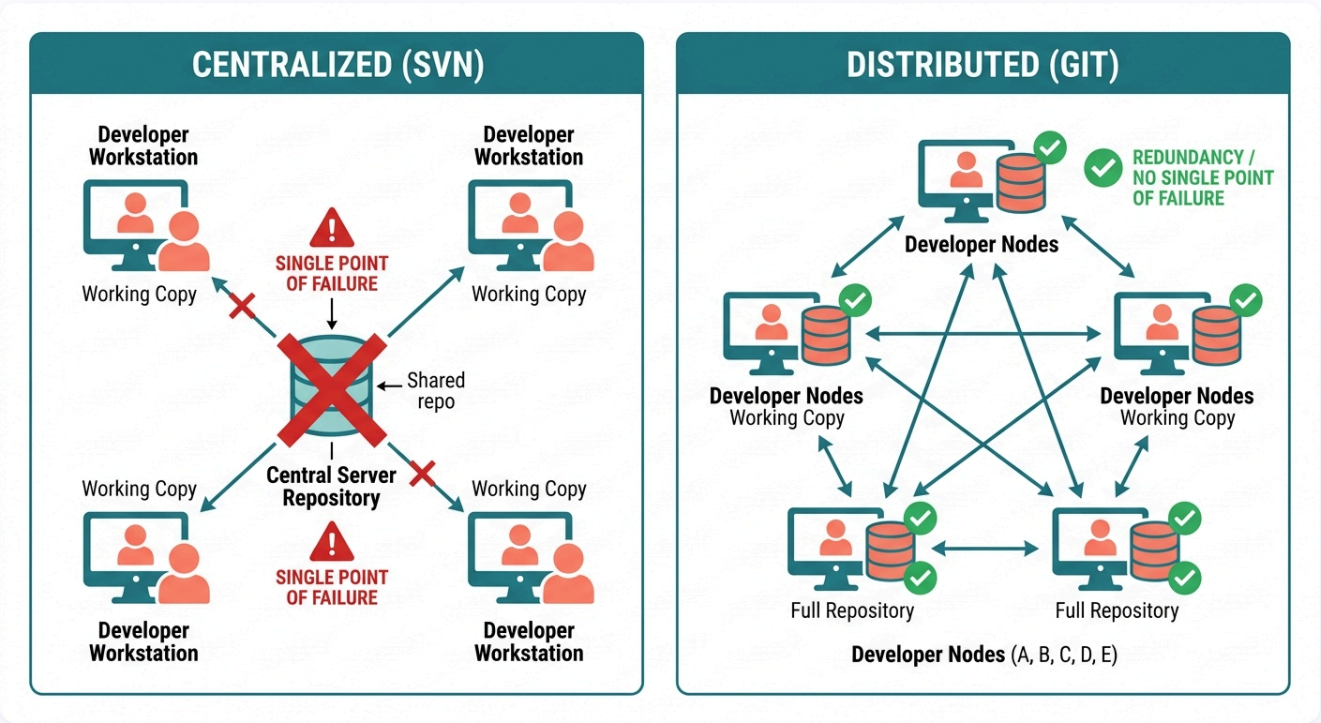
Centralized VCS (e.g., Subversion)

- **Single server** holds complete history
- Developers "check out" files, commit back
- Requires network for most operations
- Server down = no commits possible
- **Single point of failure** — server disk fails, history is lost

Distributed VCS (e.g., Git)

- Every developer has a **full clone** of the repository
- **Works offline** — commit, branch, diff locally
- Server down? Continue working, sync later
- No single point of failure

Centralized vs. Distributed — Comparison



src: ai-generated

Aspect	Centralized (SVN)	Distributed (Git)
Repository	Single server copy	Full clone on every machine
Network	Required for most operations	Only for push/pull
Server down	No commits possible	Continue working locally
Speed	Slower (network needed)	Faster (local operations)
Branching	Heavyweight, discouraged	Lightweight, encouraged

3. Introduction to Git

Snapshots, not deltas

What Is Git?

- Most popular distributed VCS in the world
- **Easy branching and merging** — nearly instantaneous
- **Data integrity** — SHA-1 checksums on everything
- **Performance** — most operations are local and fast
- **Open source** — created by Linus Torvalds (2005)

Snapshots, Not Deltas

Delta-based (SVN)

- Stores **changes** per file
- Each version = previous + delta

Snapshot-based (Git)

- Stores **complete snapshot** per commit
- Unchanged files linked (not duplicated)

Delta (SVN):

V1: FileA(v1), FileB(v1)

V2: FileA(delta), FileB(delta)

Snapshot (Git):

V1: FileA(v1), FileB(v1)

V2: FileA(v2), FileB(→v1)

↑ new

↑ linked

4. Repositories

Creating and cloning repos

What Is a Repository?

- A collection of files + **complete revision history**
- **Commits** are snapshots with unique SHA-1 hashes
- **Branches** are pointers to specific commits
- Contains both current state AND full history

git init — New Repository

```
cd my-project  
git init
```

- Creates hidden `.git` directory inside project
- `.git` contains: object database, branch pointers, config
- **Never manually edit** files inside `.git`
- After init: empty repo with no commits

git clone — Existing Repository

```
# Clone from GitHub
git clone https://github.com/username/repo.git

# Clone into a specific directory
git clone https://github.com/username/repo.git my-directory

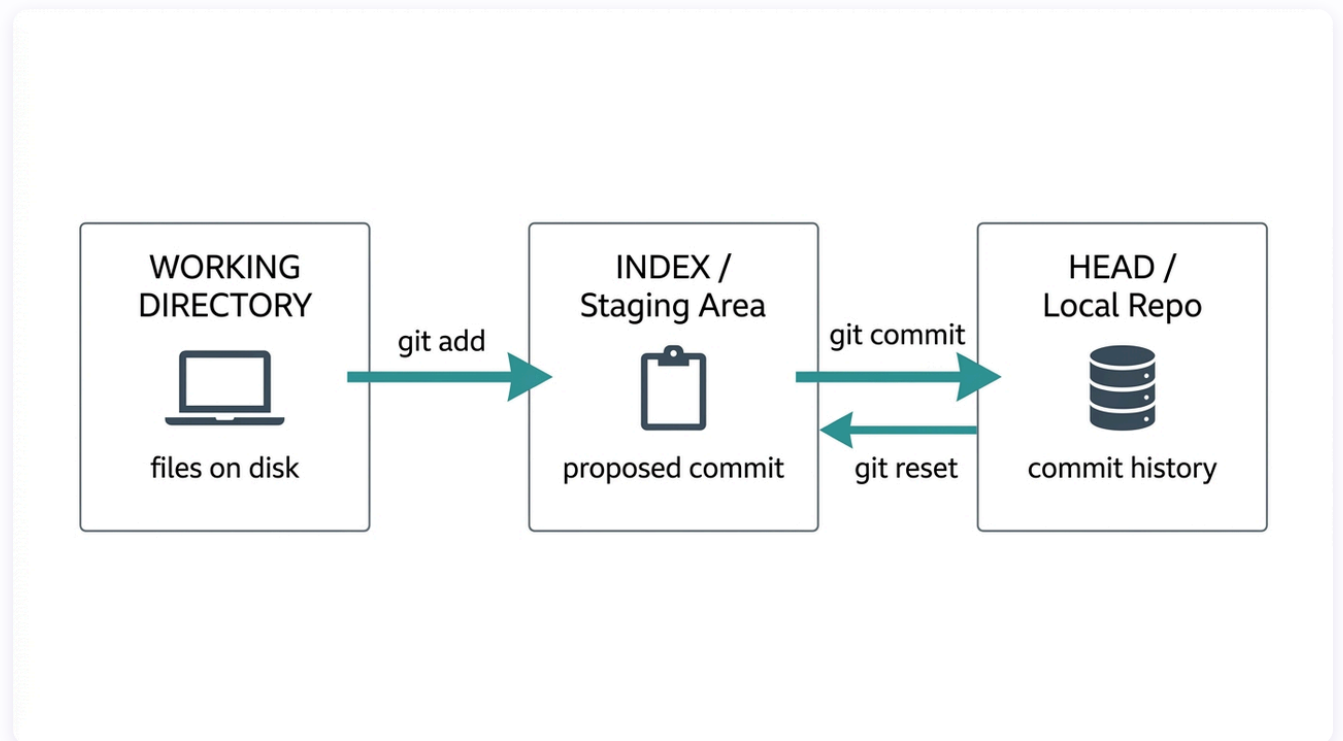
# Clone a specific branch
git clone -b develop https://github.com/username/repo.git
```

5. The Three Areas of Git

HEAD, INDEX, and Working Directory

The Three Areas

Area	Also Called	Purpose
HEAD	Local Repository	Last commit — snapshot for comparison
INDEX	Staging Area	Proposed next commit
Working Directory	Workspace	Actual files on disk



src: ai-generated

How Files Move Between Areas

```
Working Directory  ---git add--->  INDEX (Staging)  ---git commit--->  HEAD (Repo)
                  <---git reset--
```

- `git add` — copies file from Working Dir → INDEX
- `git commit` — takes INDEX snapshot → HEAD
- `git reset` — copies HEAD version → INDEX (unstage)

6. The Basic Workflow

add, commit, reset

git add — Staging Changes

```
# Stage a specific file
```

```
git add file.txt
```

```
# Stage multiple files
```

```
git add file1.txt file2.txt
```

```
# Stage all changes in current directory
```

```
git add .
```

```
# Stage all tracked files (not untracked)
```

```
git add -u
```

git commit — Recording Snapshots

```
# Commit with a message
git commit -m "feat: add user login page"

# Commit all tracked changes (skip staging)
git commit -a -m "fix: resolve null pointer in parser"
```

Each commit has:

- Unique **SHA-1 hash** (e.g., `a1b2c3d4`)
- **Author** (name and email)
- **Timestamp** and **message**
- Pointer to **parent commit(s)**

Writing Good Commit Messages

Good examples

```
feat: add user authentication with JWT  
fix: resolve crash when input is empty  
docs: update README with install steps  
test: add unit tests for calculator
```

Bad examples

```
fix  
done fixing  
update files  
asdf
```

Format

```
type: short description (≤50 chars)
```

```
Optional body with more detail.  
Explain what and why, not how.
```

Commit discipline

- Each commit = **one logical change**
- Do not combine unrelated changes
- Commit often on personal branches

git reset — Unstaging Changes

```
# Unstage a specific file (keep changes in working dir)
git reset file.txt
```

```
# Unstage all files
git reset
```

```
# DANGEROUS: discard ALL uncommitted changes
git reset --hard
```

7. Inspecting Your Repository

status, log, diff

git status — Your Dashboard

```
$ git status
On branch master

Changes to be committed:
  new file:   lect/05/05-git.key

Changes not staged for commit:
  modified:   lect/05/05-git.key

Untracked files:
  assn/01/code/quest.csv
```

git log — Commit History

```
# Example: readable commit graph
$ git log --oneline --graph --all
```

```
* 8cbcc5 feat: add login form
| * eee285f fix: resolve auth bug
|/
* 1cb431f init: first commit
```

```
# Full log includes metadata
$ git log -1
```

```
commit 8cbcc5...
Author: student <student@...>
Date:    ...
```

```
    feat: add login form
```

```
# Full log
git log
```

```
# Compact history
git log --oneline
```

```
# Recent commits
git log -5
```

```
# Branch graph
git log --oneline --graph --all
```

```
# Patch view
git log -p
```

git diff — What Changed?

```
# Unstaged changes (Working Dir vs. INDEX)  
git diff
```

```
# Staged changes (INDEX vs. HEAD)  
git diff --staged
```

```
# All uncommitted changes (Working Dir vs. HEAD)  
git diff HEAD
```

8. Branching

Lightweight pointers for parallel development

What Is a Branch?

- A **lightweight pointer** to a commit
- Default branch: `master` or `main`
- Creating a branch = creating a 40-byte pointer
- Nearly **instantaneous**, minimal disk usage

Creating and Switching Branches

```
# List all branches (* = current)
git branch

# Create a new branch (does not switch)
git branch feature/login

# Switch to an existing branch
git checkout feature/login

# Create AND switch (one step)
git checkout -b feature/login

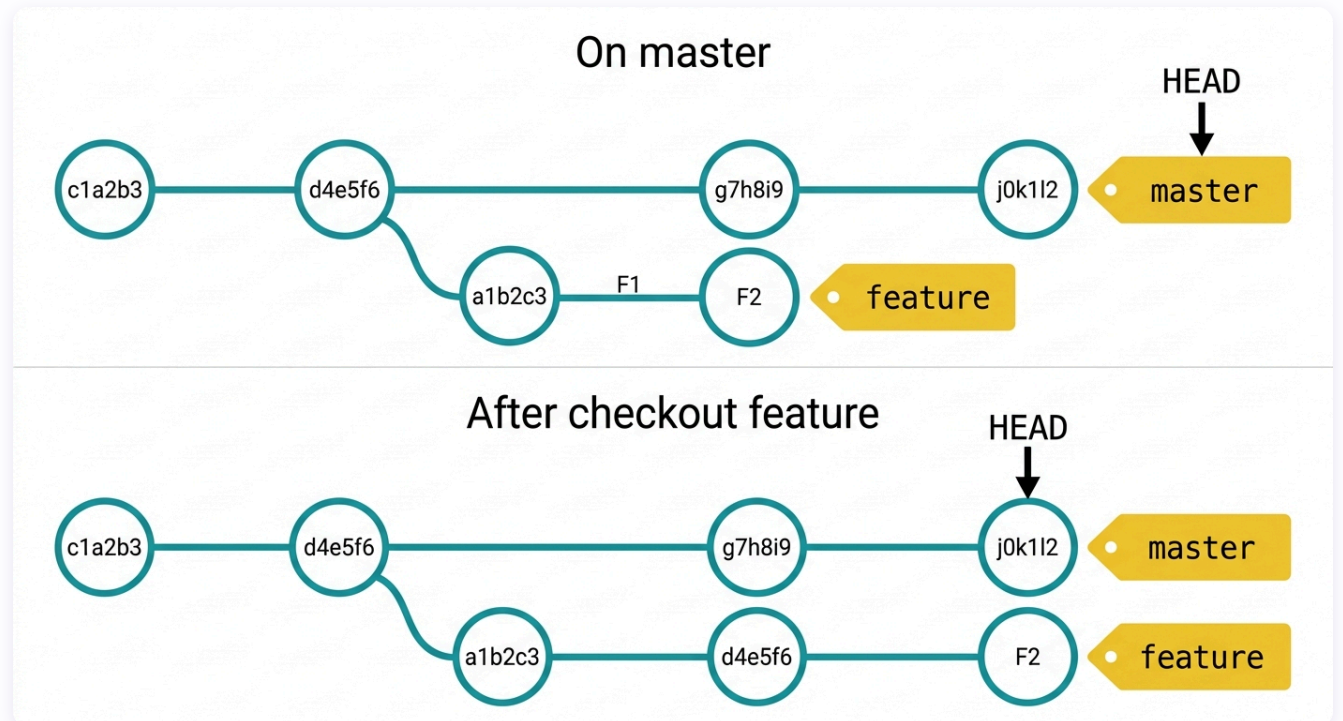
# Modern alternative (Git 2.23+)
git switch feature/login
git switch -c feature/login    # create + switch
```

Understanding HEAD

- **HEAD** = "where you are now"
- Points to current branch/commit
- Moves when you checkout/switch

```
Checkout master:
  o----o (feature)
 /
o----o----* (HEAD→master)

Checkout feature:
  o----* (HEAD→feature)
 /
o----o----o (master)
```



src: ai-generated

9. Remote Repositories

Collaborating through push and pull

What Is a Remote?

- A version of your repo hosted on **another machine**
- Default remote name: **origin**
- Enables collaboration — push and pull changes

```
# Show configured remotes
git remote -v

# Add a remote
git remote add origin https://github.com/user/repo.git
```

git push — Upload Local Commits

- Uploads commits to remote
- Only works if local is **ahead**
- If remote has new commits, pull first

```
# Push current branch  
git push
```

```
# Push specific branch  
git push origin master
```

```
# Push + set upstream tracking  
git push -u origin feature/login
```

git pull — Download Remote Changes

`git pull` = `git fetch` + `git merge`

Three scenarios:

1. No remote changes — already up to date
2. Remote ahead, no conflicts — fast-forward
3. Both have new commits — merge (may need conflict resolution)

```
# Pull from tracked branch  
git pull
```

```
# Pull from specific remote  
git pull origin master
```

10. The .gitignore File

Excluding files from version control

.gitignore — What to Exclude

Rules

- One pattern per line
- `#` starts a comment
- `*` matches any characters
- `/` at end = directory only
- `!` negates (force-include)

```
# Compiled
*.o
*.class
*.pyc
# Build
build/
dist/
node_modules/
# IDE
.vscode/
.idea/
# Secrets
.env
*.key
# OS
.DS_Store
```

Git Workflow Summary

Workspace	Staging	Local Repo	Remote Repo
---git add----->			
	---git commit----->		
---git commit -a----->			
←git reset-----			
←git diff----->			
<-----git diff HEAD----->			
		---git push----->	
		←git fetch-----	
<-----git clone/pull-----			

Git Commands Summary (1 of 2)

Command	Purpose
<code>git init</code>	Initialize a new repository
<code>git clone</code>	Clone an existing repository
<code>git status</code>	Show working tree state
<code>git log</code>	View commit history
<code>git add</code>	Stage files for next commit
<code>git commit</code>	Save staged changes to local repo
<code>git push</code>	Upload local commits to remote
<code>git pull</code>	Download and merge from remote

Git Commands Summary (2 of 2)

Command	Purpose
<code>git checkout</code>	Switch branches or restore files
<code>git branch</code>	List, create, or delete branches
<code>git diff</code>	Show differences between areas
<code>git reset</code>	Unstage files or move branch pointer

Key Takeaways

1. **Version control** prevents data loss, enables collaboration
2. Git is **distributed** — full clone, works offline
3. Git stores **snapshots**, not deltas
4. **Three areas**: Working Dir → INDEX → HEAD
5. **git add** → **git commit** → **git push** is the core workflow
6. **Branches** are lightweight pointers — use them freely
7. **.gitignore** keeps secrets and build artifacts out of history

Thank You

ICCS121 L05: Git Part 1



Q&A

All questions are welcome



EMAIL

songpon.tee@mahidol.ac.th



OFFICE HOURS

By appointment · MUIC